Interview prep Session: C#

C#

- What is boxing?
- What is unboxing?
- What is ToString() method?

OOPS

- Types of inheritance in C#
- What is abstraction?
- What is a constructor?
- What are the types of constructors?
- What is the static keyword?
- What is a partial class?
- What is the object class in C#?
- Explain the difference between **method overloading** and **method overriding** in C#. Provide real-world examples.
- How does **polymorphism behave** when a base class reference points to a derived object and both define the same method differently?
- What is the role of the virtual, override, and sealed keywords in inheritance?
- How does **runtime polymorphism** work internally in C#? Explain using **v-table** style behavior.
- What is the difference between **abstract classes** and **interfaces** in modern C# (.NET 8+)?
- Can a class inherit from multiple classes in C#? If not, how does C# support **multiple behavior contracts**?
- What happens if a derived class hides a base class method using the new keyword instead of override?
- How do **covariance** and **contravariance** relate to polymorphism in generic interfaces and delegates?
- Design a class hierarchy for a **payment system** (UPI, CreditCard, NetBanking) demonstrating inheritance and polymorphism.
- What are the risks of **deep inheritance hierarchies**, and when should **composition be preferred over inheritance**?

Data Structures

- What is the difference between **Array, List, and LinkedList** in terms of memory layout and performance?
- Explain the **time complexity** of insert, delete, and search operations for:
    - Array
    - List
    - Dictionary
    - HashSet
    - Queue
    - Stack

- When would you choose **Dictionary** over **SortedDictionary** or **ConcurrentDictionary**?
- How is a **HashSet** internally implemented, and why does it provide **O(1) average lookup**?

- What is the difference between **Stack** and **Queue**, and where would you use each in real-world applications?
- How would you implement a **custom generic Stack in C#** without using built-in collections?

- What is the use of **Span<T>** and **Memory<T>** in performance-sensitive data handling?
- Explain the difference between:
    - IEnumerable
    - ICollection
    - IList
    - IReadOnlyCollection

- How can a **poor choice of data structure** impact performance in large-scale enterprise applications?

- Design a solution in C# to **detect duplicate transactions efficiently** from a stream of 1 million records.

Records and Structs

- What is a **record** in C#, and how is it different from a traditional class?
- Explain **value equality in records** and how it differs from **reference equality in classes**.
- What is the difference between **record class** and **record struct**?
- When should you use a **struct instead of a class**? Consider memory and performance implications.
- Why is it recommended to keep **structs small and immutable**?
- What happens when a **large struct is passed to a method by value**? How can this be optimized?
- Explain the use of **with expressions** in records with an example.
- What are the **pitfalls of mutable structs** in C#?

Exception Handling & Null Safety

- What is the difference between **throw and throw ex** in C# exception handling?
- How do you **preserve stack trace** while rethrowing exceptions?
- Why should **exceptions not be used for normal control flow** in enterprise applications?
- How do **try-catch-finally blocks behave** when both catch and finally contain return statements?
- What is the difference between:
  - ApplicationException
  - SystemException
  - Custom exceptions
- How would you design a **custom exception hierarchy** for a banking or trading application?
- What happens if an **exception is thrown inside a finally block**?
- How can **exception filters (catch when)** improve error handling in C#?
- What is the difference between:
  - Nullable value types (int?)
  - Nullable reference types
- How does **enabling nullable reference types** improve code safety?
- Explain the purpose of:
  - Null-coalescing operator (??)
  - Null-coalescing assignment (??=)
- What is the difference between:
  - Null-conditional operator (?.)

    - Explicit null checks

- How do **null-forgiving operator (!)** and nullable annotations interact?
- What **compiler warnings** can occur when nullable reference types are enabled, and how should they be resolved?
- How would you safely design an **API method that may return null values**?
- Explain a **real-world bug caused by null reference handling** and how modern C# avoids it.

LINQ

- What is the difference between **IEnumerable** and **IQueryable** in LINQ?
- Explain **deferred execution** in LINQ. Why can it be both useful and dangerous?
- How can **multiple enumerations of an IEnumerable** lead to performance issues?
- What is the difference between:
    - Select
    - SelectMany
    - Where

- How do the following methods differ:
    - First()
    - FirstOrDefault()
    - Single()
    - SingleOrDefault()

- What is the difference between:
    - ToList()
    - ToArray()
    - Keeping a query **deferred**

- Explain how **GroupBy** works and where it is useful in reporting or analytics.
- How would you write a LINQ query to find the **second highest salary** from a list of employees?

- What are the **performance trade-offs** of using LINQ heavily in high-throughput systems?
- How do you **optimize LINQ queries** when working with Entity Framework or large datasets?

Tuples and Pattern Matching Enhancements

- What is the difference between **Tuple**, **ValueTuple**, and **custom DTO classes** in C#?
- When should you use **tuples** instead of defining a dedicated **class or record**?
- How do **named tuples** improve readability in method return values?
- Explain **pattern matching** in C# using:
    - is pattern
    - switch pattern
    - Relational patterns
    - Property patterns
    - Tuple patterns

- Write a **switch expression using pattern matching** to classify a transaction amount as:
    - Small
    - Medium
    - High
    - Critical

- How does **pattern matching** reduce boilerplate compared to traditional if-else chains?
- What are **discard patterns (_)**, and where are they useful?
- How can **tuple pattern matching** simplify state-based or rule-based business logic?

File Handling and I/O Operations

- What is the difference between:
    - File
    - FileInfo
    - StreamReader
    - FileStream

- When should you use:
    - using statement
    - using declarations

      while working with file I/O?

- Explain the difference between **synchronous** and **asynchronous** file operations in C#.

- How would you safely read a **very large CSV file (millions of rows)** without loading it entirely into memory?

- What are the risks of **file locking**, and how can you handle **concurrent access** to files?

- How would you write **logs to a file in a thread-safe way** in a multi-user application?

- What **exceptions** commonly occur during file I/O operations, and how should they be handled?

- How do you **serialize and deserialize objects** to/from JSON files in modern C#?

Multi-Threading

- What is the difference between:
    - Thread
    - ThreadPool
    - Task

      in C#?

- Explain **race conditions** with an example.
- How do lock, Monitor, or Mutex help prevent them?

- What is the difference between:

    - Deadlock
    - Livelock
    - Starvation

- How does **ConcurrentDictionary** differ from **Dictionary** in multi-threaded scenarios?

- How would you design a **producer-consumer system** in C# for real-time order processing?

- What is the difference between **multithreading** and **asynchronous programming** in C#?

- What happens internally when a method marked async hits an await?

- What is the difference between:
    - Task
    - Task<T>
    - ValueTask
    - void
      in async methods?

- Why is async void generally discouraged except in event handlers?

- What are common mistakes in async programming, such as:
    - Blocking on .Result or .Wait()
    - Forgetting await
    - Misusing ConfigureAwait(false)
    - Exception handling in async methods

Debugging and Tracing

- Difference between debugging and tracing?
- What is the purpose of breakpoints?
- Step Into vs Step Over vs Step Out?

- What is the Watch window used for?
- What is the Immediate window?
- What is Debug.WriteLine() used for?
- What is Trace.WriteLine()?
- What is the Call Stack window?
- How do you debug exceptions effectively?
- What are best practices for production tracing?

Secure Coding

- Why is input validation important?
- How do you prevent SQL Injection in C#?
- Why should sensitive data never be hardcoded?
- What is path traversal and how do you prevent it?
- Why should exceptions not expose internal details?
- What is the principle of least privilege?
- How should passwords be stored?
- Why is HTTPS important for .NET apps?
- What is secure deserialization awareness in .NET?
- What are secure coding best practices?

ADO.NET Fundamentals

- What is ADO.NET?
- What is the role of SqlConnection?
- What is SqlCommand used for?
- Difference between ExecuteNonQuery(), ExecuteScalar(), and ExecuteReader()?
- What is SqlDataReader?
- Connected vs disconnected architecture in ADO.NET?
- What is a DataSet?
- Why are parameterized queries important in ADO.NET?
- How do transactions work in ADO.NET?
- What are ADO.NET best practices?

SOLID Principles and Design Patterns

- What does SOLID stand for?
- What is the Single Responsibility Principle (SRP)?
- What is OCP?
- What is the Liskov Substitution Principle (LSP)?
- What is the Interface Segregation Principle (ISP)?
- What is the Dependency Inversion Principle (DIP)?
- What is the Factory Design Pattern?
- What is the Singleton Pattern and when should it be used carefully?
- What is the Repository Pattern?
- Why are SOLID principles and design patterns important?

Coding Challenges

Inheritance and Polymorphism

Payment Gateway Engine

- A fintech company supports multiple payment methods: UPI, Credit Card, and Net Banking. You need to design the system so that new payment types can be added later without changing the processing engine.

- Task: Create a polymorphic payment processing system using inheritance.

Application Crashes Due to Unhandled Exception

Scenario: Your application crashes when dividing two numbers because the denominator sometimes becomes zero. How would you handle this?

Runtime Behavior Needs to Change

Scenario: A base class Shape has a method Draw(). Different shapes must implement their own drawing logic.

Resource Must Always Be Closed

Scenario: You are reading a file and want to ensure the file is closed even if an exception occurs.

Need to Hide Implementation Details

Scenario: You want users to know that a class performs calculations but should not see how it works internally.

Data Structures

Detect Duplicate Transactions

A bank receives 1 million transaction IDs in a stream. You must detect duplicates efficiently in near O(1) lookup time.

Task

- Choose the right data structure and implement duplicate detection.

Immutable Order with Validation

- An e-commerce system receives an order. The order should be immutable after creation, and if the amount is invalid (<= 0), throw a custom exception.

Task

1. Use a record for immutable order data
2. Use custom exception handling for validation

Nullable Types and Null Safety

Safe Employee Bonus Calculator

- **Problem**: Create a method that accepts employeeName, salary, and bonusPercentage as nullable inputs and returns the final bonus safely.

- **Rules**: invalid employee name returns 'Invalid Employee'; salary defaults to 0; bonusPercentage defaults to 5%.

- **Interviewer expects**: Use nullable-safe checks, string.IsNullOrWhiteSpace, and ?? operator.

Safe Order DTO Mapper

- **Problem**: Map an Order object with nullable OrderId, CustomerName, and Amount into a display string using safe defaults.

- **Interviewer expects**: Use null-safe DTO mapping and defensive coding.

Collections and LINQ

Student Ranking System with LINQ

- Problem: Create a student list, filter marks >= 60, sort by marks descending then name ascending, and return top 3 students.

- Interviewer expects: Use Where, OrderByDescending, ThenBy, and Take.

Department-wise Employee Grouping

- Problem: Group employees by department and print employee count and average salary per department.

- Interviewer expects: Use GroupBy, Count, Average, and projection.

Scenario: Safe Employee Search

An HR system stores employees. Search by department may return null department values and some employees may have null emails. You need to safely filter and return valid email addresses.

Task
Use:

- nullable reference types
- LINQ
- null-safe filtering

Tuples and Pattern Matching Enhancements

Login Validation Using Tuple Return

- Problem: Validate username and password and return (bool IsSuccess, string Message).

- Interviewer expects: Use value tuples and validation rules.

Payment Processor Using Pattern Matching

- Problem: Process payment based on runtime type: decimal amount, string transaction ID, null, or unsupported type.

- Interviewer expects: Use switch expression, type patterns, and when guards.

Fraud Risk Classifier

A transaction system needs to classify transactions based on:

Amount international flag

Rules:

- amount < 10,000 → Low
- amount 10,000–50,000 → Medium
- amount > 50,000 and international → High
- amount > 50,000 and domestic → Medium-High

File Handling and I/O Operations

Log File Writer

- Problem: Create app.log, append 3 log entries with timestamps, then read and print the content.
- Interviewer expects: Use File.AppendAllText and File.ReadAllText.

CSV Reader and Salary Sum

- Problem: Read a CSV file of employee salaries and calculate total salary.
- Interviewer expects: Use File.ReadAllLines, Split, Skip, and LINQ aggregation.

Large Log File Reader

A production server generates a 2 GB log file. You must read and print only lines containing "ERROR" without loading the whole file into memory.

Task

- Read file efficiently using streaming I/O.

Multi-Threading

Thread-Safe Counter

- Problem: Create 5 tasks; each increments a shared counter 1000 times. Ensure final result is correct.
- Interviewer expects: Use Task and Interlocked.Increment for thread safety.

Producer-Consumer with lock

- Problem: Simulate a producer adding 10 items to a shared queue and a consumer removing them safely.

- Interviewer expects: Use Queue, lock, and Task coordination.

Parallel Report Download Engine

A reporting dashboard needs to download 3 reports from remote APIs. The reports are independent and should run concurrently to reduce total wait time.

Task

- Use async/await and Task.WhenAll().

Asynchronous Programming with Async/Await

Async File Reader

- Problem: Create a file and read it asynchronously using ReadAllTextAsync().
- Interviewer expects: Use async Main and await for file I/O.

Parallel API Simulation with Task.WhenAll

- Problem: Simulate 3 API calls using Task.Delay() and fetch them in parallel.
- Interviewer expects: Use Task.WhenAll for concurrent async execution.

Debugging and Tracing

Debug a Null Reference Bug

- Problem: Fix code so it does not throw NullReferenceException when name is null.

- Interviewer expects: Add null checks and defensive handling.

Add Trace Logging to Salary Processor

- Problem: Create a salary calculation method and add Debug.WriteLine and Trace.WriteLine.

- Interviewer expects: Use diagnostics to improve observability.

Production Error Traceability

A microservice occasionally fails during order processing. You need to: log debug messages in development.

Task

Show basic use of:

- Debug.WriteLine
- Trace.WriteLine
- correlation ID

Secure Coding

Secure Login Input Validation

- Problem: Validate username and password: username alphanumeric only, password length = 8, reject null/empty.

- Interviewer expects: Use Regex allowlist validation.

Prevent SQL Injection with Parameterized Query

- Problem: Create an ADO.NET method that fetches a user by ID securely using parameters.

- Interviewer expects: Use SqlCommand with parameters to prevent SQL injection.

ADO.NET Fundamentals

Insert Employee Record

- Problem: Insert employee name and salary into Employees table and print rows affected.
- Interviewer expects: Use SqlConnection, SqlCommand, and ExecuteNonQuery.

Read Employees with SqlDataReader

- Problem: Read all employees from Employees table and print Id, Name, and Salary.
- Interviewer expects: Use ExecuteReader and forward-only reading.

SOLID Principles and Design Patterns

Apply SRP to Invoice Processing

- Problem: Refactor invoice processing so one class calculates totals and another class prints the invoice.
- Interviewer expects: Apply Single Responsibility Principle and separation of concerns.

Factory Pattern for Notification Service

- Problem: Create INotificationService, EmailNotificationService, SmsNotificationService, and a factory method.
- Interviewer expects: Use interface-based design and Factory Pattern.